

```

classDiagram
    class NotificationService
    class MasterSchedule
    class User
    class Velocity
    class Question
    class Survey
    class Course
    class StorageManager
    class Report
    class Schedule
    class Plan
    class Material
    class FrozenSchedule
    class Checklist
    class Unit
    class TimeSlot
    class Task
    class Dimension

    NotificationService ..> MasterSchedule : Use
    MasterSchedule --> User : 1
    User --> Velocity : 1
    User --> Question : 1
    User --> Survey : 1
    User --> Course : 1
    User --> StorageManager : Use
    Survey --> Question : 1..*
    Survey --> Report : Use
    Course --> Report : 0..*
    Course --> Plan : 1
    Course --> Material : 1
    Schedule --> Plan : 1
    Schedule --> FrozenSchedule : 0..*
    Plan --> Checklist : 1
    Plan --> Task : 1
    Checklist --> Task : 1
    Task --> Unit : 1
    Task --> Dimension : 1
    Unit --> Material : 1
    Unit --> Unit : 1
    TimeSlot --> Task : 1
    TimeSlot --> FrozenSchedule : 1
    FrozenSchedule --> Plan : 1
    
```

Course

Course
+ name + study_material + plan + weekly_reports[] + final_report + status {RO} + delay {RO}
+ set_material(*args) -> None + set_plan(*args) -> None + add_task(content: Unit, velocity: Velocity) -> None + remove_task(task_ID: str) -> None + generate_report(**weekly: bool) -> None

Plan

Plan
+ schedule + legacy[] + checklist + threshold + start_date + due_date + is_strict + target_daily_time + weekly_reports[] + final_report
+ add(task_ID: str) -> None + remove(task_ID: str) -> None + check_delay() -> TimeDelta + reschedule(reason: str) -> None + update() -> None # copy unfinished task to next day

CheckList

CheckList
+ tasks[] + is_done {RO}
+ add(content: Unit, velocity: Velocity) -> str + finish(task_id: str, duration: TimeDelta) -> None + remove(task_id: str) -> None + is_finish(task_id: str) -> bool + get_finished() -> list[Task] + get_unfinished -> list[Task]

Task

Task
+ id {RO} + content_ref #expected to be a leaf node + status + approx_duration + duration + completed_at
+ finish(duration: TimeDelta) -> None + abandon() -> None

FrozenSchedule

FrozenSchedule
+ start_date {RO} + end_date {RO} + slots[] {RO} + comment {RO}
+ diff(schedule: FrozenSchedule) -> str

TimeSlot

TimeSlot
+ date + tasks[] + is_empty {RO}
+ add(task_id: str) -> None + remove(task_id: str) -> None + shift(time_span: TimeDelta) -> None + copy_to(slot: TimeSlot) -> None

Schedule

Schedule
+ start_date {RO} + end_date + slots[] + rest_days[]
+ snapshot() -> FrozenSchedule + add(task_ID: str) -> None + remove(task_ID: str) -> None + move(task_ID: str, new_date: Date) -> None + shift(slot: TimeSlot, time_span: Timedelta) -> None + extend(duration: Timedelta) -> None + redistribute_unfinished(reason: str) -> None + today_tasks() -> list[Task]

StudyMaterial

StudyMaterial
+ title + author + category + root_unit

Unit

Unit
+ topic + is_leaf {RO} + branches[] + dimension {RO}
+ branch(new_branch: Unit) -> None + has_branch(branch: Unit) -> bool + prune(branch: Unit) -> Unit + rename(new_name: str) -> None + flatten() -> list[Unit]

Dimension

Dimension
+ volume {RO} + uom {RO}

Report

Report
+ course {RO} + time_span {RO} + tasks[] {RO} + avg_velocity {RO} + complete_rate {RO}

User

User
+ first_name + last_name + name_reversed + velocity + courses[]
+ create_course(name)

Velocity

Velocity
+ pph # pages per hour + wpm # words per minute + fps # frame per second

MasterSchedule

MasterSchedule
+ collect() -> Schedule

Survey

Survey
+ questions[] + avg_velocity
+ recommend_delta() -> dict + answer(question_id: str, choice: int) ->None

Question

Question
+ id + text + choices[] + reply
+ answer(choice: int) -> None